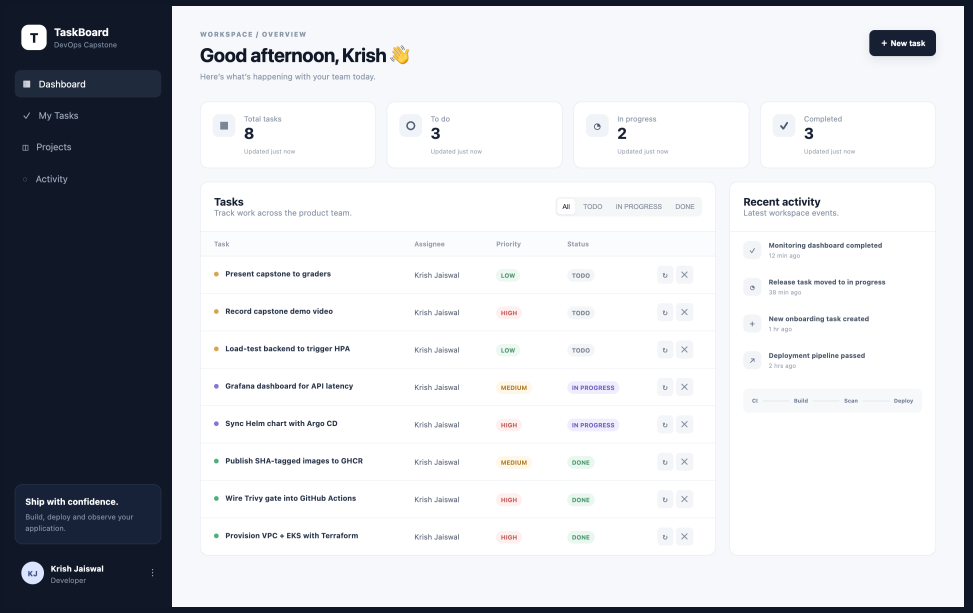
DEVSECOPS CAPSTONE

TaskBoard

Shipping a full-stack app from a laptop to a secured, monitored, autoscaled Kubernetes cluster on AWS



React · FastAPI · PostgreSQL · Docker · GitHub Actions · Trivy · Terraform · EKS · Helm · Argo CD · Prometheus · Grafana

Krish Jaiswal · October 2026

One app, every layer automated

TaskBoard is a SaaS-style task manager: a React dashboard, a FastAPI REST API and PostgreSQL.

The point is the journey: every commit is tested, built into a non-root image, scanned for CVEs, published with its commit SHA, deployed by Helm, synced by Argo CD and observed in Grafana.

16

pytest tests passing on an isolated in-memory DB

0

fixable HIGH / CRITICAL CVEs in both images (Trivy)

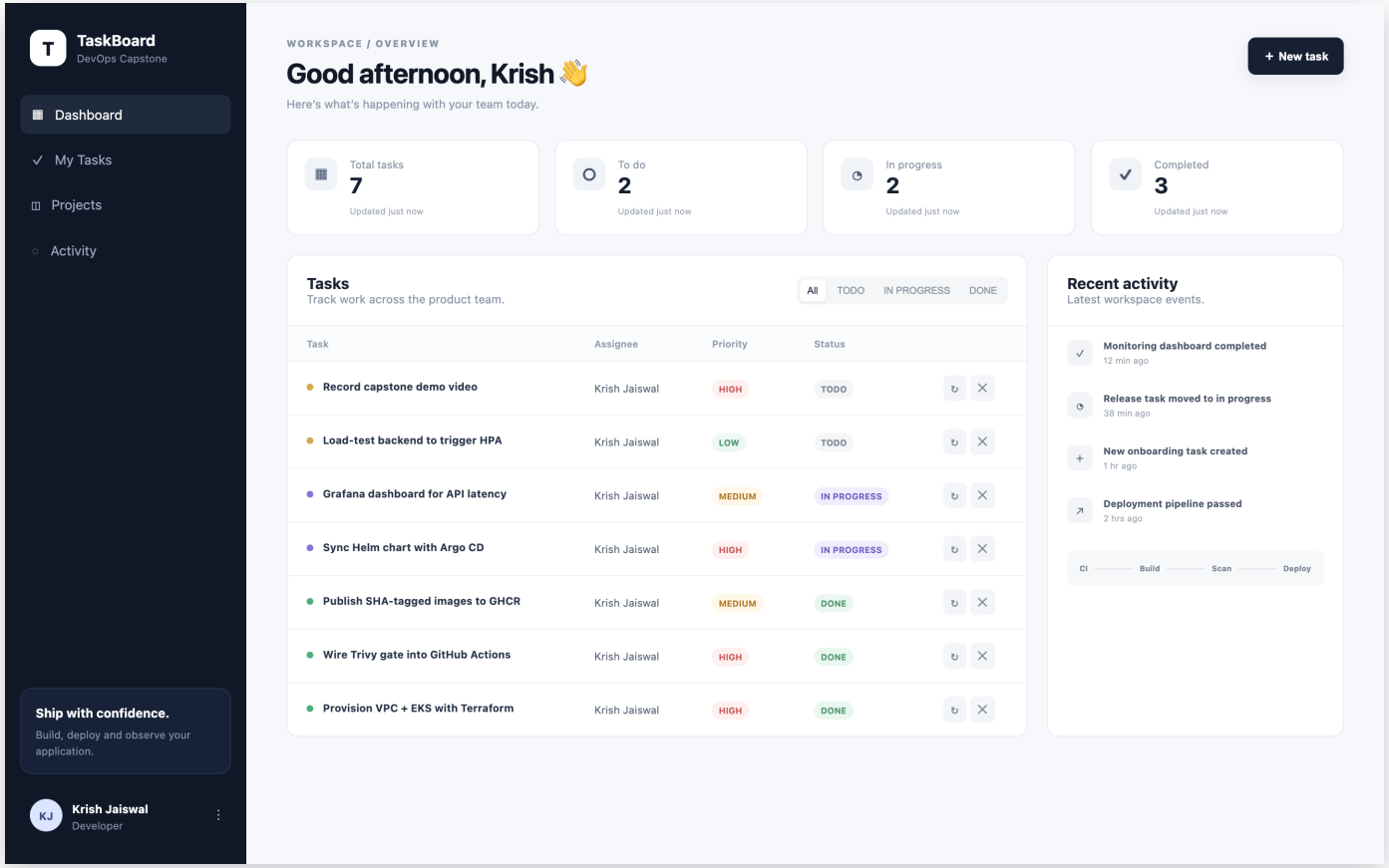
61

AWS resources in the Terraform plan (VPC + EKS)

2 → 4

backend pods, autoscaled by the HPA under load

The application: React + FastAPI + PostgreSQL



GET	`/api/tasks` list, newest first
GET	`/api/tasks/{id}` one task (404 if missing)
POST	`/api/tasks` create, validated
PUT	`/api/tasks/{id}` partial update
DELETE	`/api/tasks/{id}` remove (204)
GET	`/api/tasks/stats` KPI card counts
GET	`/health` · `/ready` · `/metrics` probes + Prometheus

Schema owned by Alembic · migrations advisory-locked so replicas can't race

Tech stack

Application

React 19 · Vite 8
FastAPI · SQLAlchemy 2.1
PostgreSQL 16 · Alembic

Quality

pytest 9 · TestClient
in-memory SQLite
16 tests, 7 endpoints

Containers

multi-stage Dockerfiles
non-root (uid 10001 / 101)
Docker Compose

CI/CD

GitHub Actions
GHCR, SHA-tagged images
kind smoke deploy

Security

Trivy 0.75 image gate
Pod Security Admission
dropped capabilities

Infrastructure

Terraform · AWS VPC
EKS 1.35 · node group
EBS CSI · Pod Identity

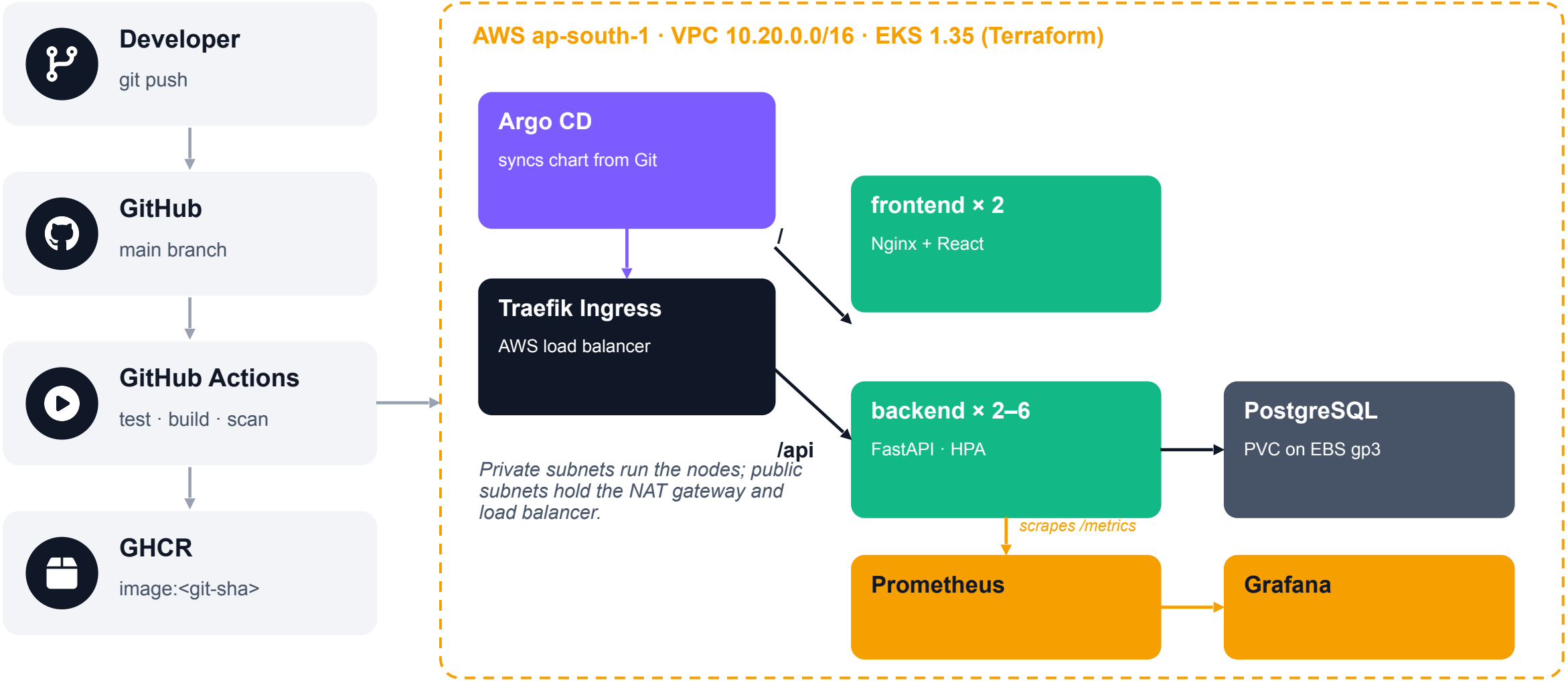
Delivery

Helm chart · Traefik Ingress
HPA · metrics-server
Argo CD (GitOps)

Observability

Prometheus Operator
ServiceMonitor
Grafana dashboard

Architecture



CI/CD: four gated stages on every push to main

1 Test

pytest -v (16 tests)
npm ci && vite build

Fails when any test fails

2 Image

build per component
Trivy HIGH/CRITICAL gate
push to GHCR :<sha>

Fails when a fixable CVE is found

3 Deploy test

ephemeral kind cluster
helm upgrade --install
full CRUD smoke test

Fails when pods aren't Ready or an API call fails

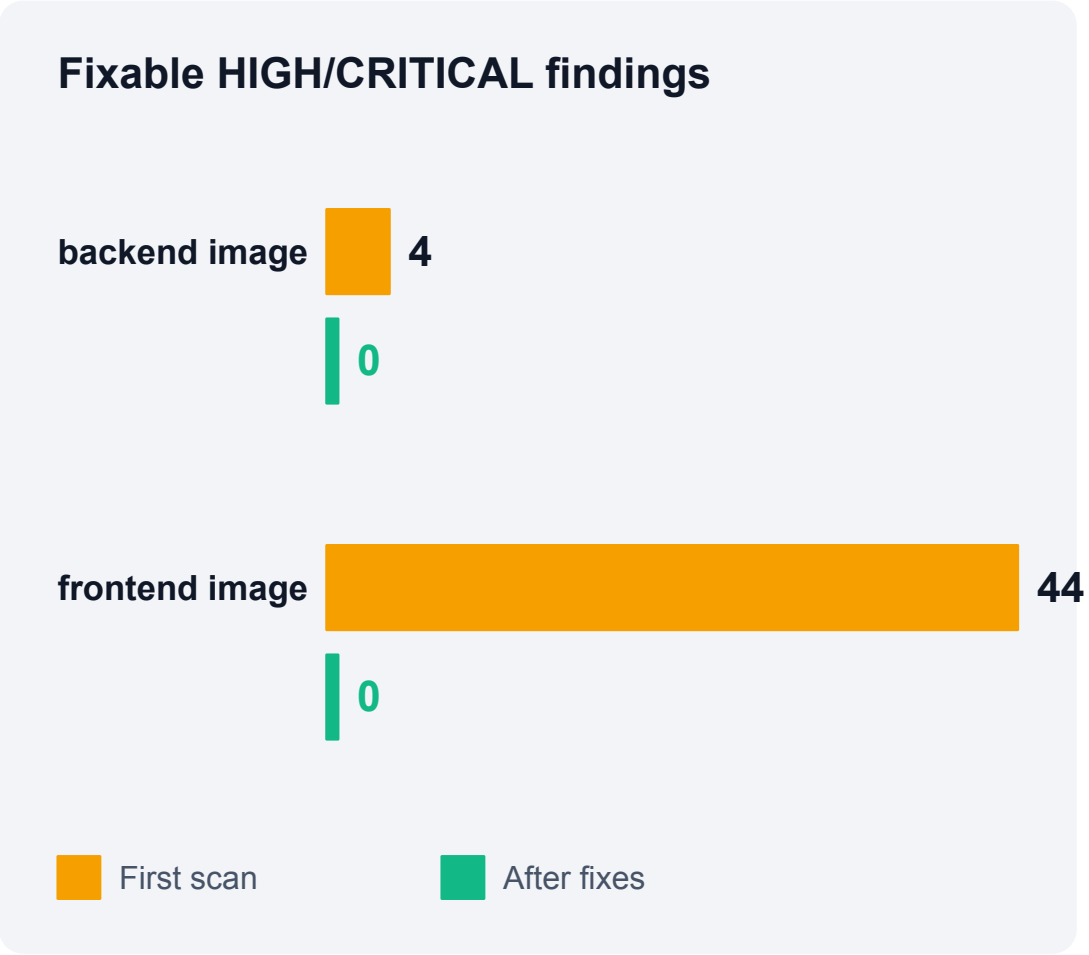
4 GitOps

write <sha> into values-prod.yaml
[skip ci] commit

→ Argo CD syncs EKS

Image tag = commit SHA, never latest `ghcr.io/venkat1701/taskboard-backend:<sha>` → `git show <sha>`

DevSecOps: Trivy blocked our first build, and was right



The CVE we explained

urllib3 CVE-2026-97687 (HTTPS-proxy traffic interception) plus msgpack and setuptools findings — none were our dependencies. They were vendored inside `pip`, which the API never runs in production.

- Backend: multi-stage build, venv copied into a clean image with pip removed
- Frontend: apk upgrade on the unprivileged Nginx base cleared 44 fixed Alpine CVEs
- --ignore-unfixed keeps the gate actionable: fail only when a fix exists

Containers: small, non-root, health-gated

backend image

351 MB

- python:3.12-slim → deps into /opt/venv
- clean runtime stage: apt upgrade, no pip
- runs as appuser (uid 10001)
- HEALTHCHECK on /health
- alembic upgrade head → uvicorn

frontend image

112 MB

- node:22-alpine → npm ci && vite build
- nginx-unprivileged:1.29-alpine runtime
- runs as nginx (uid 101) on :8080
- BACKEND_URL rendered by envsubst
- SPA fallback + /api reverse proxy

`docker compose up --build`



postgres

pg_isready → healthy



backend

migrations + /health → healthy

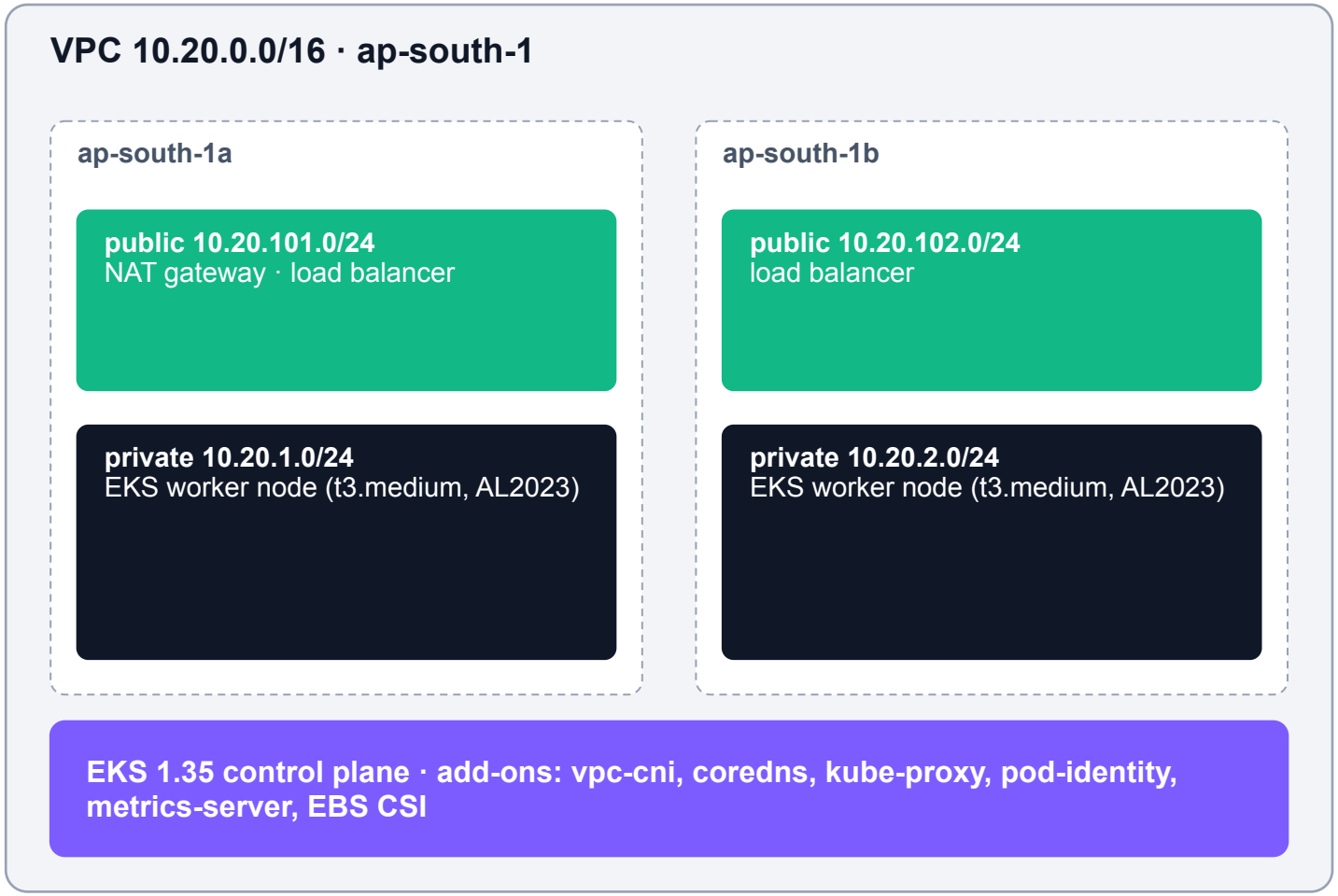


frontend

Nginx :8080 → healthy

depends_on: condition: service_healthy — no start-up races

Terraform: VPC + EKS as code



61

resources to add
0 to change, 0 to destroy
(terraform plan)

Why these choices

- EKS 1.31 is no longer offered; 1.35 is in standard support
- AL2 AMIs don't exist for ≥ 1.33 → AL2023 nodes
- EBS CSI driver, or the Postgres PVC never binds
- metrics-server, or the HPA has no CPU data
- Single NAT gateway ≈ $0.27/h total: apply, demo, destroy

Kubernetes + Helm: one chart, every environment

```
$ kubectl get pods -n taskboard
NAME                                READY   STATUS
taskboard-…-backend-7b8c…-k26s6     1/1     Running
taskboard-…-backend-7b8c…-x2vst     1/1     Running
taskboard-…-frontend-759b…-hgsvz    1/1     Running
taskboard-…-frontend-759b…-zcdhz    1/1     Running
taskboard-…-postgres-66ff…-w6rh7    1/1     Running

$ kubectl get ingress -n taskboard
CLASS    HOSTS                PATHS
traefik  taskboard.localhost  /api → backend, / → frontend

$ helm list -n taskboard
taskboard  rev 3  deployed  taskboard-1.1.0
```



Deployments

backend 2–6 (HPA) · frontend 2 · probes: startup, / ready, /health



Hardened pods

runAsNonRoot, seccomp RuntimeDefault, no privilege escalation, caps dropped



PostgreSQL

Recreate strategy for RWO volume, PGDATA subdir, PVC kept on uninstall



Ingress + HPA

Traefik routes / and /api; HPA on 60 % CPU, Deployment omits replicas

GitOps with Argo CD: Git is the source of truth



`selfHeal`

a manual kubectl edit is reverted to what Git says

`prune`

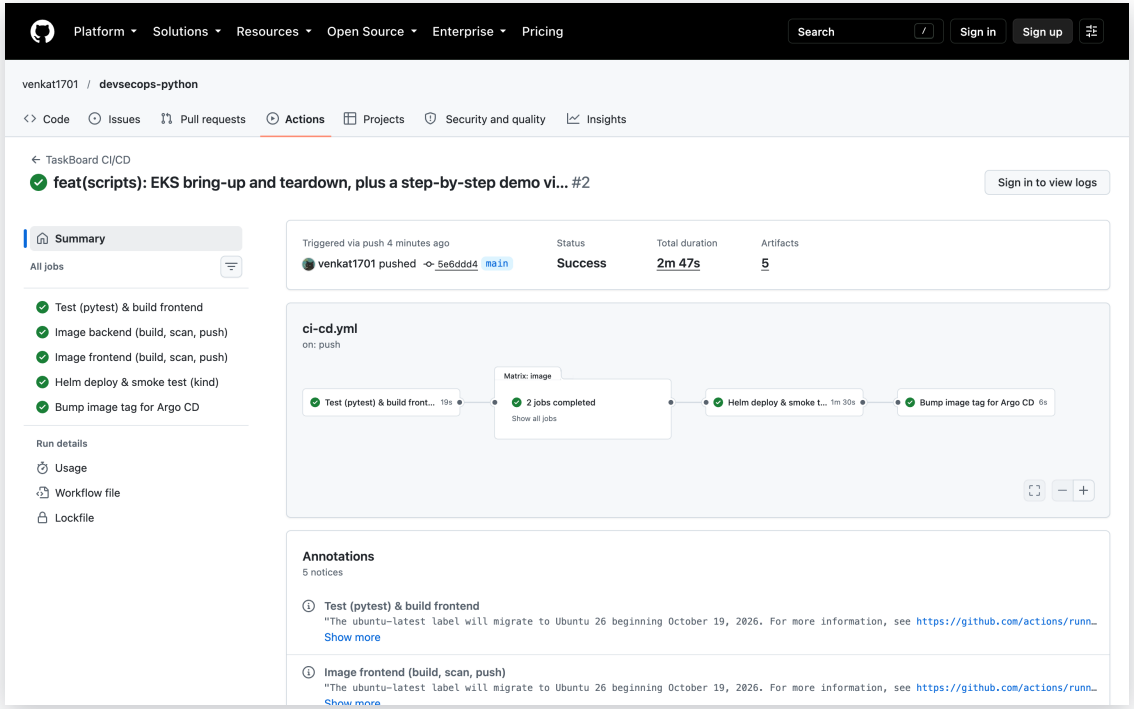
resources deleted from Git are deleted from the cluster

`rollback`

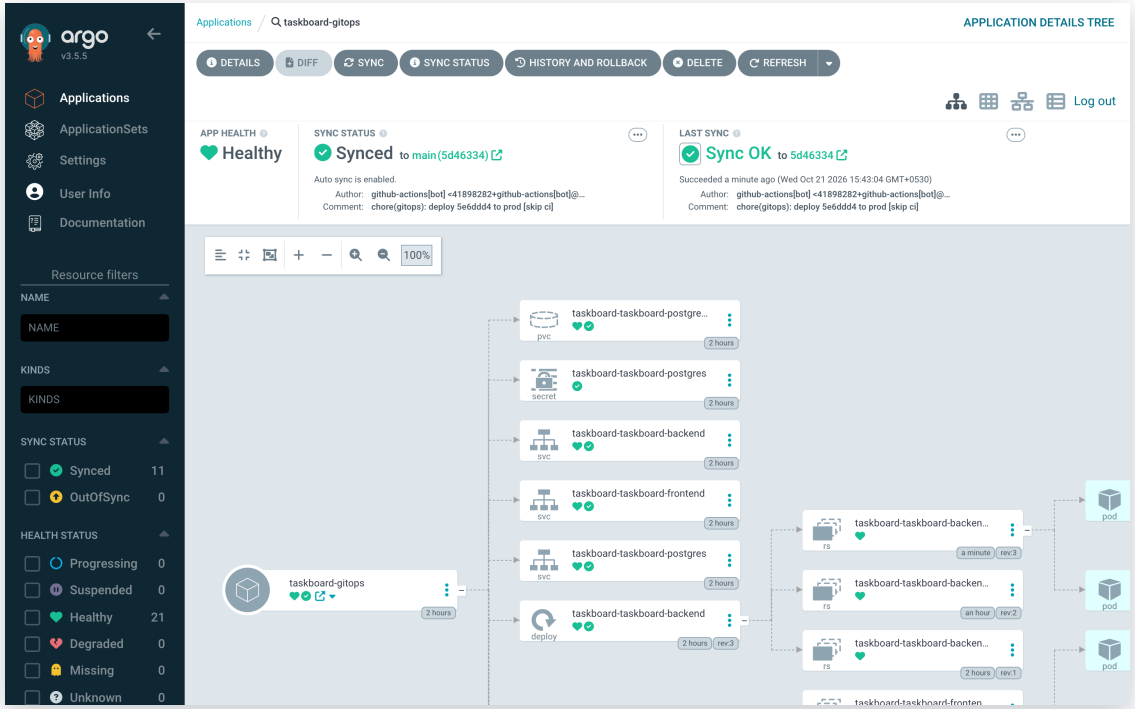
git revert the tag commit; Argo CD does the rest

argocd/application.yaml · argocd/install.sh

Proof: green pipeline, Argo CD in sync

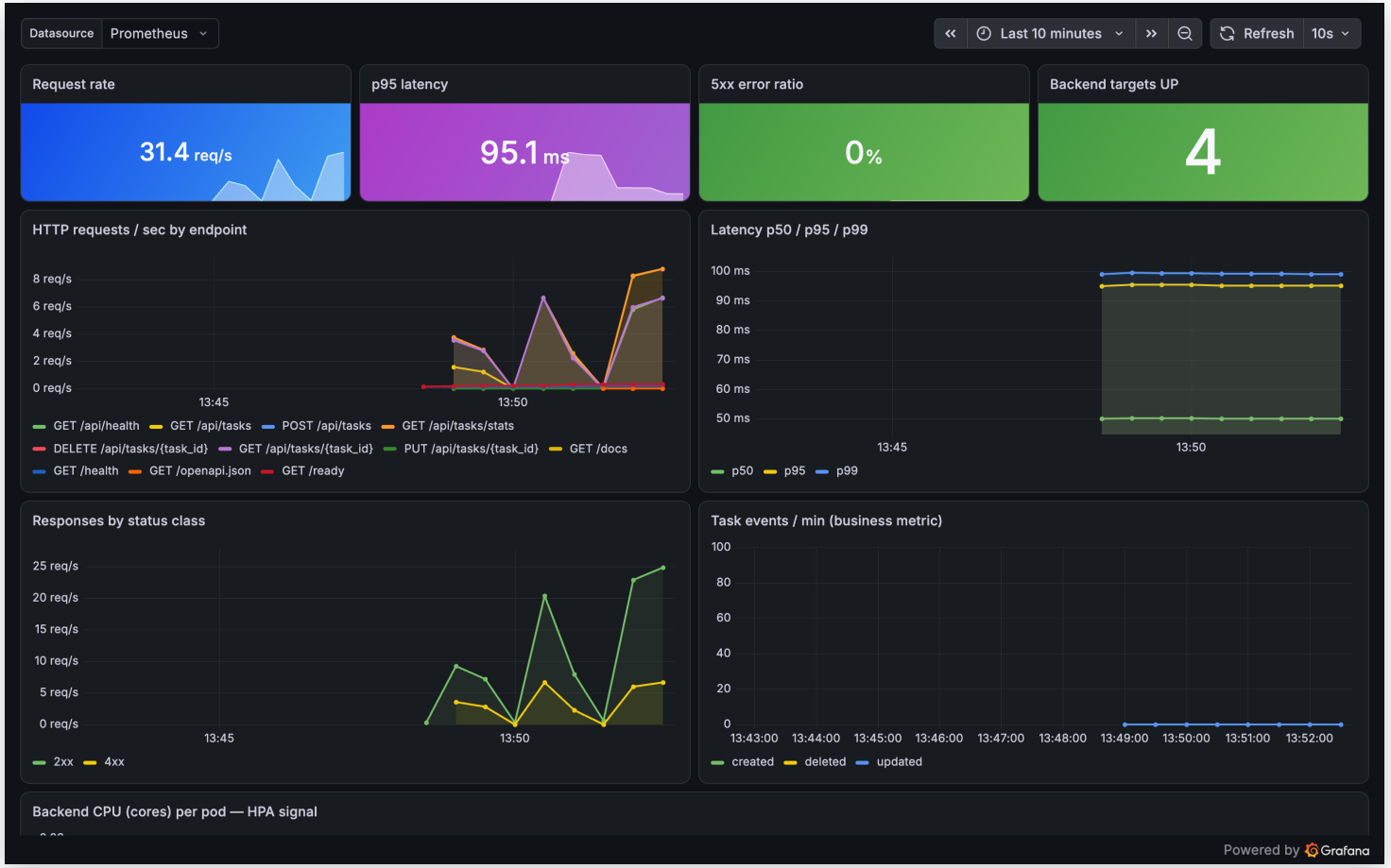


GitHub Actions: all 5 jobs green; images pushed to GHCR tagged with the commit SHA



Argo CD: Synced + Healthy at the bot's chore(gitops) commit; manual scale-down reverted in ~6 s

Observability: Prometheus + Grafana



Live panels: request rate, p95 latency, 5xx ratio, targets UP, per-endpoint traffic, status classes, CPU per pod

/metrics

request count + latency histograms, and a business counter taskboard_task_events_total

ServiceMonitor

Prometheus Operator discovers every backend pod automatically — 4/4 targets UP after scaling

Dashboard as code

Grafana JSON ships in the Helm chart; the sidecar loads it, no clicking

Autoscaling in action: 2 → 4 pods under load

2 — **baseline**

```
New size: 2; replicas below Spec.MinReplicas
```

3 — **+4m45s**

```
New size: 3; cpu utilization above target
```

4 — **+6m31s**

```
New size: 4; cpu utilization above target
```

How it works

- metrics-server reports pod CPU against the 100m request
- HPA targets 60 % average utilisation, min 2, max 6
- scripts/load-test.sh drives /api traffic through the Ingress
- Prometheus picked up each new pod as a target with no config change
- Scale-down waits 5 minutes (stabilisation window)

kubectl get events -n taskboard --field-selector reason=SuccessfulRescale

Making it work: 12 defects found and fixed

Terraform HCL did not parse

multi-attribute one-line blocks

EKS 1.31 no longer offered

moved to 1.35 + AL2023 nodes

Ingress → wrong Service and port

taskboard-backend:8080 didn't exist

Nginx proxied to host 'backend'

templated BACKEND_URL per environment

Pipeline never pushed to GHCR

Docker Hub, :latest tags, 3 rebuilds

Trivy could never fail the build

added --exit-code 1

Frontend ran Nginx as root

unprivileged image on :8080

Init container waited forever

pg_isready as uid 10001: pass -U/-d

Create-task modal never closed

React currentTarget is null after await

Postgres initdb fails on EBS

lost+found → PGDATA subdirectory

Compose start-up races

health-gated depends_on

ingress-nginx retired (2026)

switched to Traefik

Key learnings and next steps

Learnings

- Run every layer for real; reading YAML hides broken wiring
- Gates must be able to fail: a scanner without --exit-code is decoration
- Liveness ≠ readiness: a DB outage should drain pods, not restart them
- Pin versions and keep them current: EKS, AMIs and Ingress controllers retire
- Git as the deployment API makes rollback a git revert

Next steps

- Amazon RDS instead of in-cluster Postgres
- External Secrets + AWS Secrets Manager
- cert-manager TLS on the Ingress
- S3 remote state with locking for Terraform
- App authentication; alerting rules in Alertmanager

Live demo

Pipeline run · Terraform plan · Argo CD sync · kubectl · Prometheus targets · Grafana

- git push → GitHub Actions green
- terraform plan / AWS console
- Argo CD: Synced + Healthy
- kubectl get pods,svc,ingress,hpa
- Prometheus /targets: UP
- Grafana: live panels under load

Thank you · Krish Jaiswal